Programación por bloques con Scratch

Nombre: _____ Fecha: _____

Qué vas a aprender

- Conocer el entorno de Scratch: escenario, objetos, bloques y área de programas
- Programar movimientos, apariencia y sonidos con eventos
- Usar bloques de control, sensores y mensajes
- Crear y compartir una animación o un juego sencillo

Para pensar

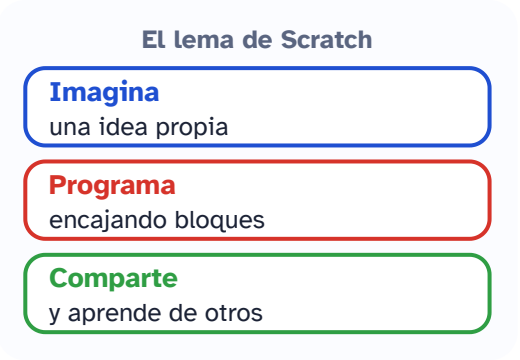
¿Se puede programar un videojuego sin escribir una sola línea de código?

Explicación

1.1 ¿Qué es Scratch?

Scratch es un lenguaje de programación visual creado en 2007 por el grupo *Lifelong Kindergarten* del **MIT** (Instituto Tecnológico de Massachusetts). Es gratuito y funciona en el navegador o instalado en el ordenador.

En vez de escribir código, se arrastran y encajan bloques. Cada bloque tiene una forma que indica dónde puede ir, así que es imposible cometer errores de sintaxis (comas o paréntesis olvidados). Con Scratch se pueden crear historias interactivas, animaciones, juegos y simulaciones, y compartirlas con una comunidad de millones de personas.



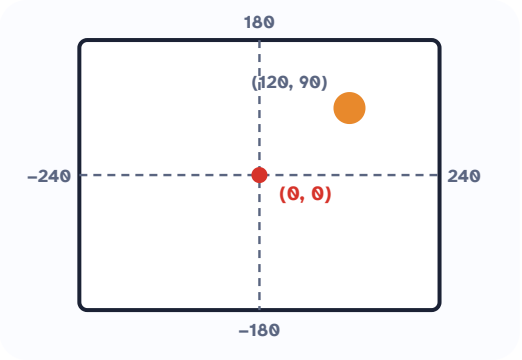
1.2 El entorno de trabajo

- **Escenario**: el espacio donde ocurre todo. Mide 480 × 360 píxeles y usa coordenadas: el centro es (0, 0), x va de −240 a 240 y y de −180 a 180.
- **Objetos** (*sprites*): los personajes y elementos. Cada uno tiene sus propios programas, **disfraces** (aspectos) y sonidos.
- **Fondos**: las imágenes del escenario.
- **Paleta de bloques**: organizada por categorías de colores.
- **Área de programas**: donde se encajan los bloques.

La bandera verde inicia el programa y la señal de stop lo detiene.

Cada objeto tiene tres pestañas: **Código**, donde se encajan los bloques; **Disfraces**, con un editor de dibujo (vectorial o de mapa de bits) para crear o modificar sus imágenes; y **Sonidos**. Los programas pertenecen a cada objeto: si un script no funciona, lo primero es comprobar que está en el objeto correcto. El

proyecto se guarda en la nube con una cuenta o se descarga al ordenador como archivo .sb3.



1.3 Las categorías de bloques

Categoría	Color	Ejemplos
Movimiento	azul	mover 10 pasos, girar, ir a x: y:, deslizar
Apariencia	morado	decir, cambiar disfraz, mostrar, esconder, tamaño
Sonido	rosa	tocar sonido, cambiar volumen
Eventos	amarillo	al hacer clic en la bandera, al presionar tecla, enviar mensaje
Control	naranja	esperar, repetir, por siempre, si... entonces
Sensores	azul claro	¿tocando...?, preguntar, cronómetro
Operadores	verde	+, −, número al azar, y, o, no
Variables	naranja oscuro	crear variable, sumar a, listas

Además se pueden crear **bloques propios** y añadir **extensiones** (lápiz, música, micro:bit...).



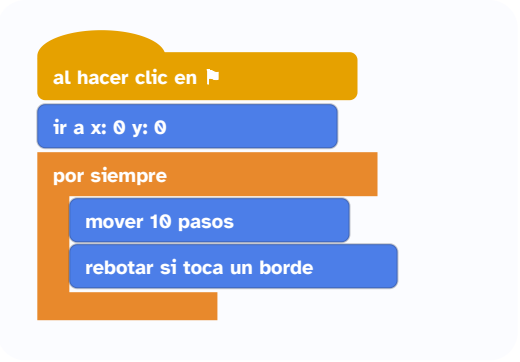
2.1 Eventos: cuándo empieza cada programa

Todo programa en Scratch empieza con un **bloque de evento** (con forma de «sombrero»). El evento indica cuándo se ejecutan los bloques que cuelgan debajo:

- «al hacer clic en la bandera verde»: al iniciar.
- «al presionar la tecla espacio»: al pulsar una tecla.
- «al hacer clic en este objeto».
- «al recibir mensaje»: cuando otro objeto envía una señal.

Un objeto puede tener varios programas a la vez, cada uno con su evento, y todos se ejecutan **en paralelo**. Es la base de la **programación orientada a eventos**, como funcionan las apps y los videojuegos.

Un mismo objeto puede tener varios scripts que empiezan con el mismo evento; al pulsar la bandera verde se ejecutan todos **a la vez**, en paralelo. Esto es muy útil (uno mueve el objeto y otro lo anima), pero exige cuidado: si dos scripts cambian la misma variable o la misma posición, pueden interferir. El botón rojo de **detener** para todos los scripts.



2.2 Movimiento y apariencia

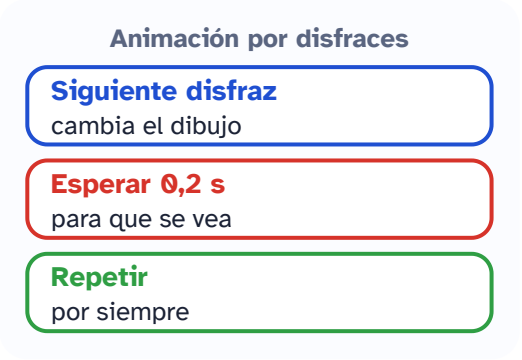
Con los bloques de **movimiento** los objetos se desplazan por el escenario: «mover 10 pasos» avanza en la dirección a la que apunta; «ir a x: y:» lo coloca en una posición exacta; «deslizar en 1 segundo a...» lo mueve suavemente.

Con los de **apariencia** cambian de aspecto. La **animación** se consigue alternando disfraces con una pequeña espera, igual que los fotogramas del cine:

```
por siempre
    siguiente disfraz
    esperar 0.2 segundos
```

Si además el objeto se mueve mientras cambia de disfraz, parece que camina. Los bloques «decir» y «pensar» muestran bocadillos como en un cómic.

Los bloques de **apariencia** cambian cómo se ve el objeto: «cambiar a disfraz», «siguiente disfraz» (para animar alternando dibujos con una pequeña espera), «decir Hola durante 2 segundos», «cambiar tamaño», «fijar efecto color», «mostrar» y «esconder». También el escenario tiene sus bloques: «cambiar fondo a» permite pasar de la portada al juego y del juego a la pantalla final. El orden de las capas (delante o detrás) decide qué objeto se ve encima.



2.3 Sonido y música

Los proyectos cobran vida con **sonido**. Cada objeto tiene su pestaña de sonidos, donde se pueden elegir de la biblioteca, grabar con el micrófono o subir archivos. Los bloques «iniciar sonido» (que continúa con el programa mientras suena) y «tocar sonido hasta que termine» (que espera a que acabe) permiten añadir efectos: un salto, una moneda, una explosión. Con «fijar volumen» y los efectos de tono se puede variar cómo suenan.

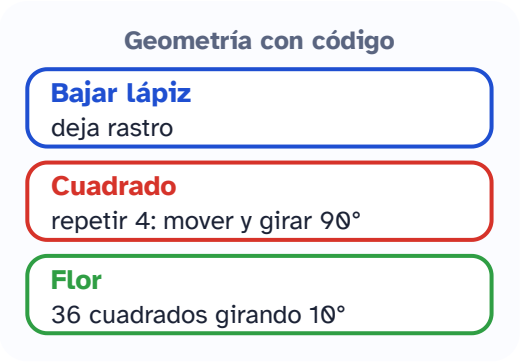
La extensión **Música** añade bloques para tocar notas e instrumentos: «tocar nota 60 durante 0,25 pulsos» toca un do central; cambiando el número se obtienen otras notas (62 es re, 64 es mi…). Combinando notas, esperas y bucles se programan melodías, y con eventos de teclado se puede construir un **piano** o una batería. Un consejo: la música de fondo suele ir en el escenario dentro de un «por siempre» con «tocar sonido hasta que termine», para que se repita sin cortes.



2.4 El lápiz: dibujar con código

La extensión **Lápiz** convierte a cualquier objeto en un pincel: con «bajar lápiz», el objeto deja un rastro al moverse, y con «subir lápiz» deja de dibujar. Se pueden cambiar el color y el grosor, y «borrar todo» limpia el escenario. Es una forma excelente de practicar bucles y geometría: «repetir 4 veces: mover 100 pasos, girar 90°» dibuja un cuadrado, y para cualquier polígono regular de n lados el giro es 360/n grados.

Combinando bucles anidados y pequeños cambios se obtienen figuras sorprendentes: si se repite 36 veces «dibujar un cuadrado y girar 10°» aparece una flor geométrica; si en cada vuelta se cambia un poco el color, el dibujo se llena de degradados. El bloque «sellar» imprime la imagen del objeto en el fondo, útil para dibujar estrellas en un cielo o huellas. El lápiz también sirve para hacer gráficas: un objeto que sube y baja según un dato va trazando su evolución.



3.1 Control y sensores: interacción

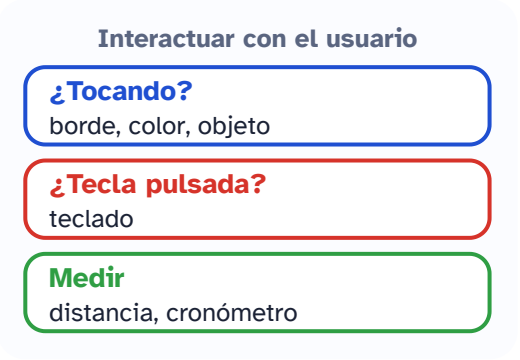
Los bloques de **control** organizan el flujo: **repetir**, **por siempre**, **esperar** y **si... entonces... si no**. Los **sensores** responden preguntas sobre lo que pasa: ¿estoy tocando el borde?, ¿se ha pulsado una tecla?, ¿a qué distancia está el ratón?

Combinados, permiten crear juegos. Por ejemplo, mover un personaje con las flechas:

```
al hacer clic en ⚑
por siempre
    si ¿tecla flecha derecha presionada? entonces
        cambiar x en 5
    si ¿tocando Enemigo? entonces
        decir "¡Ay!" durante 1 segundos
```

El bucle «por siempre» comprueba continuamente las condiciones, decenas de veces por segundo.

Los sensores también miden: «distancia a», «posición x del ratón», «cronómetro» o «volumen del sonido» (con el micrófono). Combinados con condicionales permiten proyectos muy variados: un personaje que huye cuando el ratón se acerca, un juego que se acaba a los 60 segundos o una animación que reacciona cuando damos una palmada.

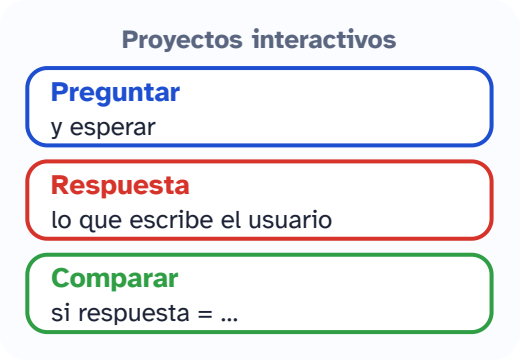


3.2 Preguntar y responder: proyectos interactivos

El bloque «**preguntar … y esperar**» muestra una pregunta y un cuadro de texto en el que el usuario escribe. Lo que escribe se guarda en el bloque redondo «**respuesta**», que se puede comparar con un condicional o guardar en una variable. Así se programan cuestionarios, calculadoras, conversaciones con un personaje o juegos de adivinanzas.

Un concurso típico: preguntar «¿Cuál es la capital de Portugal?» y esperar; si respuesta = «Lisboa», decir «¡Correcto!» y sumar 1 a puntos; si no, decir «La respuesta era Lisboa». Para que acepte «lisboa» en minúsculas, Scratch no distingue mayúsculas en las comparaciones de texto. Con operadores de texto

(«unir», «letra 1 de», «longitud de») se pueden construir frases con el nombre del jugador: «Hola, » unido a respuesta. Guardando preguntas y respuestas en **listas**, un mismo script sirve para cualquier número de preguntas.



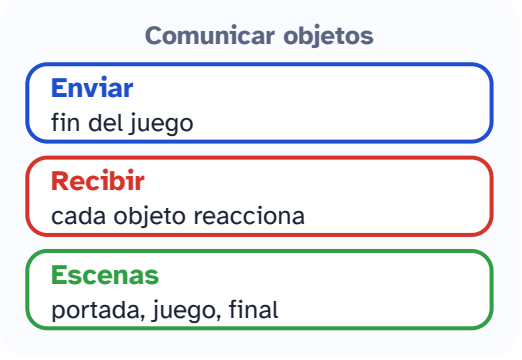
3.3 Mensajes: que los objetos se comuniquen

En un proyecto con varios objetos, a menudo uno tiene que avisar a otro. Para eso se usan los **mensajes**:

- «**enviar** mensaje *fin del juego*»: lanza una señal a todos los objetos.
- «**al recibir** *fin del juego*»: el objeto que escucha ejecuta su programa (por ejemplo, el escenario cambia al fondo de «Game Over»).

Los mensajes también sirven para contar historias en orden: el personaje A habla y, al terminar, envía «turno de B»; B lo recibe y responde. Así se coordinan escenas sin depender de esperas cronometradas, que suelen descuadrarse.

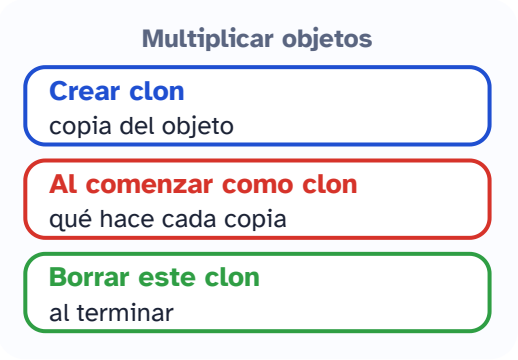
Los mensajes son la mejor forma de organizar un juego en **escenas**: un mensaje «empezar» oculta la portada y muestra a los personajes; «nivel 2» cambia el fondo y aumenta la velocidad de los enemigos; «fin del juego» detiene todo y muestra la puntuación. Conviene dar a los mensajes nombres que expliquen qué ocurre.



3.4 Clones: muchos objetos a la vez

En muchos juegos hacen falta muchos objetos iguales: gotas de lluvia, disparos, enemigos, estrellas. En lugar de crear cincuenta objetos a mano, se usan los **clones**. El bloque «crear clon de mí mismo» hace una copia del objeto, con sus mismos disfraces y programas, y el evento «al comenzar como clon» indica qué debe hacer cada copia: aparecer en una posición aleatoria, caer, moverse. Cuando ya no hace falta, «borrar este clon» lo elimina.

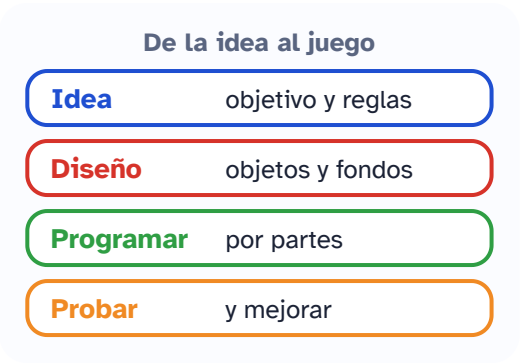
Un ejemplo: el objeto «gota» se esconde al empezar y, por siempre, crea un clon cada 0,2 segundos. Cada clon aparece en x aleatoria y arriba del todo, se muestra, baja hasta el suelo y se borra. Las variables «solo para este objeto» son distintas para cada clon, lo que permite que cada enemigo tenga su propia energía. Scratch permite hasta 300 clones a la vez, por eso hay que borrarlos cuando salen del escenario; si no, el proyecto se vuelve lento o deja de crear clones nuevos.



4.1 De la idea al proyecto

1. **Idea** y descomposición: ¿qué objetos hay?, ¿qué hace cada uno?
2. **Diseño**: dibuja o elige los objetos, disfraces y fondos.
3. **Programa por partes**: primero el movimiento del protagonista, luego los enemigos, después los puntos.
4. **Prueba** cada parte con la bandera verde antes de seguir.
5. **Depura**: si algo falla, haz clic en un bloque suelto para ejecutarlo y ver qué hace.
6. **Comparte** el proyecto con instrucciones y créditos.

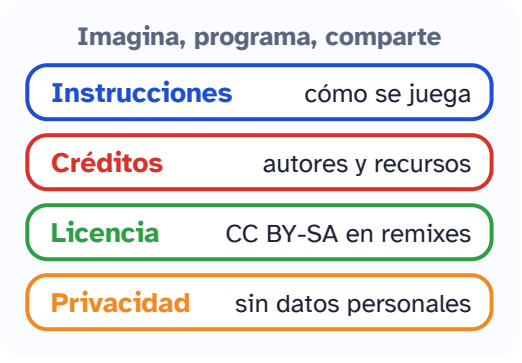
En la comunidad de Scratch puedes ver el código de otros proyectos y hacer una **reinvención** (*remix*), citando siempre al autor original.



4.2 Compartir, remezclar y comentar

Al **compartir** un proyecto en la web de Scratch, cualquiera puede verlo, jugarlo, dejar comentarios y hacer un **remix**. Todos los proyectos compartidos tienen una licencia **Creative Commons BY-SA**: se pueden reutilizar citando al autor y compartiendo la nueva versión con la misma licencia. Scratch añade automáticamente un enlace al proyecto original en los remixes, pero es de buena educación mencionarlo también en las notas y explicar qué se ha cambiado.

Una buena página de proyecto incluye **instrucciones** claras (qué teclas usar, cuál es el objetivo) y **notas y créditos** (quién ha hecho el proyecto, qué imágenes o sonidos son de otros). La comunidad tiene normas: ser respetuoso, dar críticas constructivas, no compartir datos personales (nombre completo, dirección, colegio, fotos) y no subir contenido ajeno sin permiso. Los **estudios** reúnen proyectos sobre un tema, y el profesorado puede crear cuentas de clase para compartir solo dentro del grupo.



4.3 Más allá de Scratch

La programación por bloques es una puerta de entrada. Los mismos conceptos (eventos, bucles, condicionales, variables) aparecen en los lenguajes escritos como **Python** o **JavaScript**.

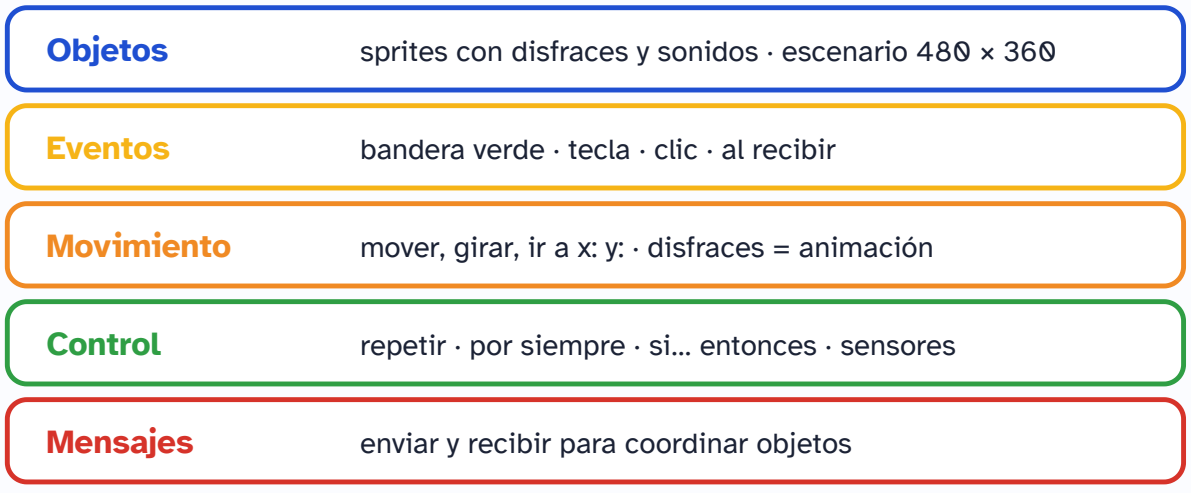
- **MakeCode** (Microsoft) programa placas micro:bit y juegos con bloques y permite ver el código equivalente en JavaScript o Python.
- **App Inventor** (MIT) crea aplicaciones para móviles con bloques.
- **Code.org** y **Blockly** ofrecen retos y la tecnología de bloques en la que se basan muchas herramientas.

Lo que aprendes con Scratch no se pierde: es la lógica de cualquier programa.

Lo que se aprende con Scratch se traslada directamente: un «por siempre» es un while True en Python, un «si… entonces» es un if y una variable es una variable. Muchos estudiantes dan el salto con entornos híbridos, que muestran a la vez los bloques y el código de texto equivalente, como MakeCode (con JavaScript y Python) o EduBlocks.



Scratch en cinco ideas



Ejercicios

1. **¿Qué categoría de bloques usarías para que un objeto diga «Hola»?**

2. **Un objeto está en (50, –20) y ejecuta «ir a x: 0 y: 0». ¿Dónde queda?**

3. **¿Qué hace este programa: repetir 4 veces → mover 100 pasos, girar 90°, con el lápiz bajado?**

4. **Escribe los bloques para que un objeto cambie de color al hacer clic sobre él.**

5. **¿Por qué un programa de colisión debe estar dentro de «por siempre»?**

6. **Explica la diferencia entre «enviar mensaje» y «enviar mensaje y esperar».**

7. **¿Qué valor de x tiene el borde derecho del escenario?**

8. **Diseña el programa de un objeto que siga al puntero del ratón.**

9. **Descompón el juego «atrapa la fruta» en objetos y programas.**

10. **Crea una historia interactiva de dos personajes que dialoguen usando mensajes.**

11. **¿Qué diferencia hay entre «iniciar sonido» y «tocar sonido hasta que termine»?**

12. **¿Qué hacen «bajar lápiz» y «subir lápiz»?**

13. **¿Dónde se guarda lo que escribe el usuario tras «preguntar y esperar»?**

14. **¿Para qué sirven los clones?**

15. **¿Qué licencia tienen los proyectos compartidos en Scratch?**

16. **Escribe tres datos que no debes compartir en tu perfil de Scratch.**

17. **Programa un saludo personalizado: pregunta el nombre y responde «Hola, [nombre]».**

18. **Dibuja un triángulo equilátero y un pentágono con el lápiz. ¿Qué giros usas?**

19. **¿Por qué hay que borrar los clones cuando salen del escenario?**

20. **Programa una música de fondo que se repita sin parar en el escenario.**

21. **Diseña un cuestionario de 3 preguntas que cuente los aciertos.**

22. **Haz que cada clon de enemigo tenga su propia energía (3) y desaparezca al llegar a 0.**

23. **¿Qué debe incluir la página de un proyecto compartido?**

24. **Explica qué hace: repetir 36: (repetir 4: mover 80, girar 90°), girar 10°.**

25. **¿Cómo harías que un personaje huya cuando el ratón se acerca a menos de 50 pasos?**

26. **Diseña un juego de disparos con clones: nave controlada con flechas, disparos con la barra espaciadora y enemigos que caen. Describe objetos y scripts.**

27. **Programa un instrumento que cambie el tono según la posición x del ratón.**

28. **Crea un generador de dibujos aleatorios con el lápiz: color, tamaño y número de lados al azar.**

29. **Haz un remix de un proyecto de la comunidad: explica qué cambiarías, cómo lo citarías y qué normas respetarías.**

30. **Compara mensajes y clones: ¿cuándo usarías cada uno? Pon un ejemplo de proyecto que necesite los dos.**

Practica

1. **Scratch fue creado por el...**

   A) Google
   B) MIT
   C) Microsoft
   D) Apple

2. **Los personajes de Scratch se llaman...**

   A) fondos
   B) variables
   C) objetos (sprites)
   D) listas

3. **El centro del escenario tiene coordenadas...**

   A) (0, 0)
   B) (240, 180)
   C) (480, 360)
   D) (–240, –180)

4. **Todo programa empieza con un bloque de...**

   A) operadores
   B) sonido
   C) eventos
   D) lápiz

5. **Para animar un objeto se alterna...**

   A) el fondo
   B) el disfraz con una pequeña espera
   C) el volumen
   D) el tamaño sin espera

6. **El bloque «por siempre» pertenece a la categoría...**

   A) control
   B) movimiento
   C) apariencia
   D) sonido

7. **«¿tocando borde?» es un bloque de...**

A) eventos
B) sensores
C) sonido
D) lápiz

8. **Para que un objeto avise a otro se usan...**
   A) disfraces
   B) fondos
   C) mensajes
   D) variables

9. **Con la programación por bloques es imposible cometer errores de...**
   A) sintaxis
   B) lógica
   C) diseño
   D) ortografía

10. **Hacer una versión propia de un proyecto ajeno citando al autor es una...**
    A) copia ilegal
    B) reinvención (remix)
    C) extensión
    D) plagio

11. **Para que el programa espere a que acabe un sonido se usa...**
    A) iniciar sonido
    B) tocar sonido hasta que termine
    C) detener sonidos
    D) fijar volumen

12. **La extensión que permite dibujar es...**
    A) Música
    B) Lápiz
    C) Traducir
    D) Vídeo

13. **Para dibujar un hexágono se gira...**
    A) 90°
    B) 60°

C) 45°

D) 120°

14. **Lo que escribe el usuario se guarda en…**

A) respuesta

B) disfraz

C) fondo

D) volumen

15. **Un clon empieza a funcionar con el evento…**

A) al hacer clic en ⚑

B) al comenzar como clon

C) al recibir

D) al presionar tecla

16. **El máximo de clones a la vez en Scratch es…**

A) 30

B) 300

C) 3.000

D) sin límite

17. **Una variable «solo para este objeto» en un clon…**

A) la comparten todos

B) es propia de cada clon

C) no existe

D) se borra sola

18. **Los proyectos compartidos en Scratch tienen licencia…**

A) ©

B) CC BY-SA

C) de pago

D) ninguna

19. **En la página de un proyecto conviene poner…**

A) la dirección de casa

B) instrucciones y créditos

C) la contraseña

D) nada

20. **La nota 60 en la extensión Música es…**

A) do central

B) la

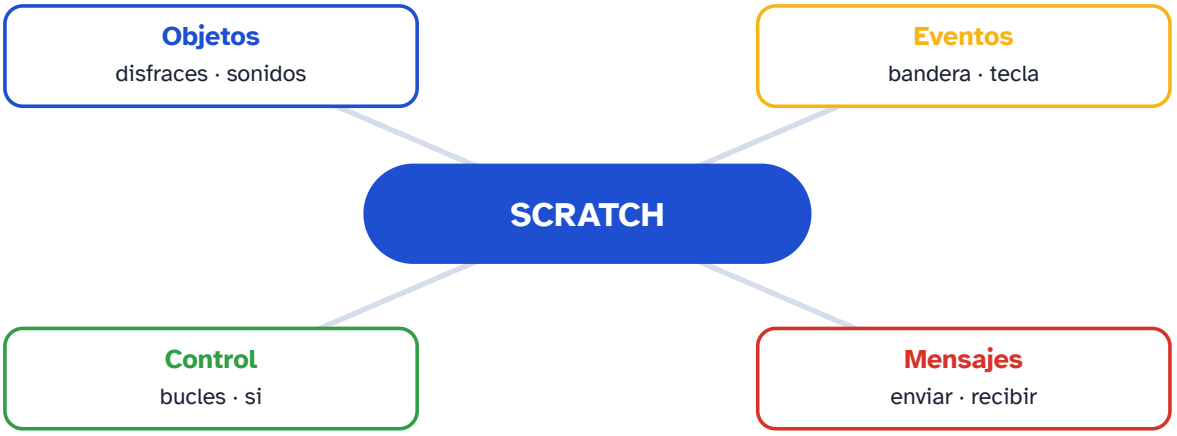
C) mi

D) sol

Explícalo con tus palabras

Incluye: Qué es la programación por bloques y su ventaja; Las partes del entorno de Scratch; Cómo se anima un objeto; Para qué sirven los mensajes.

Resumen



Glosario

Scratch	Lenguaje de programación visual por bloques del MIT.
Objeto (sprite)	Personaje o elemento del proyecto.
Escenario	Espacio de 480 × 360 donde ocurre todo.
Disfraz	Cada una de las imágenes de un objeto.
Fondo	Imagen del escenario.
Script	Grupo de bloques encajados bajo un evento.
Evento	Bloque que indica cuándo empieza un script.
Sensor	Bloque que comprueba lo que ocurre.
Mensaje	Aviso entre objetos.
Clon	Copia de un objeto en ejecución.
Extensión	Conjunto de bloques añadidos: lápiz, música, micro:bit.
Lápiz	Extensión para dibujar.
Respuesta	Lo que escribe el usuario tras preguntar.
Remix	Versión propia de un proyecto ajeno, citándolo.
Estudio	Colección de proyectos sobre un tema.
CC BY-SA	Licencia que permite reutilizar citando y compartiendo igual.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Objeto (sprite)

Personaje o elemento con sus programas, disfraces y sonidos.

Escenario

480 × 360 píxeles, centro en (0, 0).

Disfraz

Cada aspecto de un objeto; alternarlos crea animación.

Evento

Bloque sombrero que indica cuándo empieza un programa.

Por siempre

Bucle que se repite sin fin.

Sensor

Bloque que pregunta: ¿tocando?, ¿tecla presionada?

Mensaje

Señal que un objeto envía y otros reciben.

Remix

Versión propia de un proyecto ajeno, citando al autor.

Extensión Lápiz

Permite que un objeto dibuje al moverse.

Preguntar y esperar

Pide un dato; se guarda en «respuesta».

Clon

Copia de un objeto creada mientras se ejecuta el programa.

Licencia de Scratch

CC BY-SA: remezclar citando y compartiendo igual.

Soluciones

1. **B)** Grupo Lifelong Kindergarten, 2007.
2. **C)** Tienen disfraces y sonidos.
3. **A)** x de –240 a 240, y de –180 a 180.
4. **C)** Con forma de sombrero.
5. **B)** Como fotogramas.
6. **A)** Naranja.
7. **B)** Responde preguntas.
8. **C)** Enviar y al recibir.
9. **A)** Los bloques solo encajan donde pueden ir.
10. **B)** Cultura de compartir.
11. **B)** Espera al final.
12. **B)** Lápiz.
13. **B)** 360 / 6.
14. **A)** Bloque respuesta.
15. **B)** Al comenzar como clon.
16. **B)** 300.
17. **B)** Cada clon tiene la suya.
18. **B)** Se pueden remezclar citando.
19. **B)** Para que se entienda.
20. **A)** Do central.

Soluciones de los ejercicios

1. Apariencia (bloque «decir»).
2. En el centro del escenario, (0, 0).
3. Dibuja un cuadrado de 100 pasos de lado.
4. al hacer clic en este objeto → cambiar efecto color en 25.
5. Porque hay que comprobar continuamente si se tocan; fuera del bucle solo se comprobaría una vez.
6. El segundo espera a que terminen los programas que reciben el mensaje antes de continuar.
7. 240.
8. al hacer clic en ⚑ → por siempre → apuntar hacia puntero del ratón; mover 5 pasos.
9. Cesta (se mueve con flechas), fruta (cae desde arriba en x al azar; si toca la cesta, suma punto y vuelve arriba), escenario (fondo y fin de partida).
10. Respuesta abierta; debe usar enviar y al recibir para que cada personaje hable en su turno.
11. El primero empieza el sonido y el programa sigue; el segundo espera a que el sonido acabe antes de continuar.
12. Con el lápiz bajado el objeto dibuja al moverse; con el lápiz subido se mueve sin dibujar.
13. En el bloque «respuesta».
14. Para crear muchas copias de un objeto (gotas, enemigos, disparos) sin tener que hacerlas a mano.
15. Creative Commons BY-SA: reutilizar citando al autor y compartiendo igual.
16. Nombre completo, dirección, nombre del colegio, teléfono o fotos personales.
17. Preguntar «¿Cómo te llamas?» y esperar; decir (unir «Hola, » respuesta) durante 2 segundos.
18. Triángulo: repetir 3: mover 100, girar 120°. Pentágono: repetir 5: mover 100, girar 72°.
19. Porque Scratch admite un máximo de 300 clones; si no se borran, el proyecto se ralentiza y deja de crear clones nuevos.
20. Escenario: al hacer clic en ⚑: por siempre: tocar sonido «música» hasta que termine.
21. Fijar aciertos a 0. Preguntar P1; si respuesta = R1: sumar 1. Igual con P2 y P3. Al final: decir (unir «Aciertos: » aciertos).
22. Variable «energía» solo para este objeto. Al comenzar como clon: fijar energía a 3. Si toca disparo: cambiar energía en –1; si energía = 0: borrar este clon.
23. Instrucciones de uso, notas y créditos (autores, recursos usados y proyecto original si es un remix).
24. Dibuja 36 cuadrados, girando 10° entre cada uno: una flor o rosetón circular (36 · 10° = 360°).
25. Por siempre: si distancia a puntero del ratón < 50: apuntar en dirección contraria (girar 180° respecto al ratón) y mover 10 pasos.
26. Nave: se mueve con flechas; al presionar espacio: crear clon de disparo. Disparo (clon): ir a la nave, subir hasta el borde o tocar enemigo y borrarse. Enemigo: crea clones cada

segundo en x aleatoria, bajan; si tocan disparo: sumar puntos y borrarse; si tocan la nave: restar vida. Escenario: muestra «Game over» al recibir el mensaje.

27. Por siempre: si ratón presionado: tocar nota (redondear((posición x del ratón + 240) / 20) + 48) durante 0,25 pulsos. Así, de izquierda a derecha, las notas suben de 48 a 72.
28. Por siempre: ir a posición aleatoria; fijar color del lápiz a aleatorio(0, 100); n ← aleatorio(3, 8); tam ← aleatorio(20, 60); bajar lápiz; repetir n: mover tam, girar 360/n; subir lápiz; esperar 0,5 s.
29. Elegir un juego, añadir un nivel nuevo y una puntuación máxima. En notas y créditos: «Remix de [título] de [usuario]; he añadido el nivel 2 y el récord». Respetar la licencia BY-SA, no usar recursos sin permiso y no incluir datos personales.
30. Mensajes: para coordinar objetos distintos (empezar nivel, fin del juego). Clones: para muchos objetos iguales (meteoritos). Un juego de meteoritos usa clones para los meteoritos y un mensaje «fin del juego» cuando la nave pierde todas las vidas, para que todos los clones se detengan.